Module 06 — Secrets (Optional) FOR DUMMIES

Goal: stop trusting `admin`/`admin`, understand the four credential-management approaches already wired into this repo, and be able to argue for one in a design review.

Everything so far ran with zero credential setup because Containerlab bakes cEOS's defaults into the generated inventory. Great for a lab, disqualifying for anything real. This module is short because the deep material already exists — it's a guided tour with opinions.

---

1. The invariant that never changes

Whatever backend you pick, one rule holds:

> **Files that get committed reference secret *names*. The *values* live somewhere else — encrypted, or outside the repo entirely.**

Every approach below is just a different answer to "where is somewhere else, and how does it get into the running process?"

2. The four approaches, and when each wins

All three Nornir-path options work identically from the scripts' point of view: a wrapper sets `NORNIR_USERNAME`/`NORNIR_PASSWORD` in the environment, and `creds.py` (which every course script already imports) overrides the inventory if it finds them. Only the wrapper differs.

	Ceremony	Values encrypted?	Committable?	Reach for it when
`.env` (`run-with-env.sh`)	none	no	never (gitignored)	solo lab, 30-second demo
`pass` (`run-with-pass.sh`)	GPG key + store	yes (GPG)	the store can be its own git repo	you/team already live in GPG
`sops+age` (`run-with-sops.sh`)	one keypair	yes (values only — keys stay readable)	**yes, that's the point**	team sharing, GitOps/CI shape

	Ceremony	Values encrypted?	Committable?	Reach for it when
Ansible Vault (`automation/ansible/`)	vault password + collections	yes (whole file)	yes	you're on the Ansible path anyway

The full walkthroughs — installs, one-time setup, day-to-day commands — are in `automation/nornir/SECRETS.md` (first three) and the root README's Secrets section (Vault).

3. The exercise

Do the loop end-to-end with **sops + age** — it's the one closest to how real pipelines work, and the one with the most transferable concepts:

1. Follow SECRETS.md Option C: install `age` (apt) and `sops` (GitHub `.deb`), generate a keypair, copy `secrets.example.yaml` → `secrets.enc.yaml`, encrypt in place.
2. **Open the encrypted file and look at it.** Keys readable, values ciphertext. This is sops' entire value proposition over "encrypt the whole blob": the file still diffs and code-reviews.
3. Run a course script through it:

   ```
   ./run-with-sops.sh python3 03_napalm_facts.py
   ```

   `sops exec-env` decrypted into that one process's environment — nothing on disk, nothing in shell history.
4. Prove the override is real: put a wrong password in the file (`sops secrets.enc.yaml` opens it decrypted in your editor), re-run, watch auth fail. Fix it back.
5. Read the "rotating who can decrypt" section in SECRETS.md — adding a coworker's key is a recipient-list change, not a re-type of every secret. That property is what makes this approach *team-shaped*.

Then, for contrast, spend five minutes on the Ansible Vault flow from the root README (`cp vault.yml.example vault.yml`, `ansible-vault encrypt`, `--ask-vault-pass`). Same invariant, different ergonomics.

4. The honest ceiling

None of these four give you rotation, per-user audit, or short-lived credentials — that's centralized secrets-manager territory (HashiCorp Vault, CyberArk, cloud KMS backing sops instead of age). The workflow you just learned doesn't change when a team gets there; the backend behind the env vars does. Which is precisely why the invariant in section 1 is the thing to internalize, not any particular tool.

Checkpoint

`run-with-sops.sh` runs a course script with credentials the script never saw in plaintext

You can explain to a coworker why `secrets.enc.yaml` is safe to commit but `.env` never is

You can name which approach you'd propose for an Aston team repo, and defend it in one paragraph

What you learned

The reference-names-not-values invariant; four implementations of it in rising order of team-readiness; sops' diff-friendly encryption model; and where all of them hand off to real secrets infrastructure.

That's the course. You now have a fabric that deploys, configures, verifies, and overlays itself from files in version control — and you know which file to touch for any change you'd want to make. Go show someone the two-command EVPN build from module 05; that demo sells itself.